



Market Clearing as an Optimisation Problem

Author:

Hossein Farahmand

Department of Electric Energy, NTNU

Date:

October 2024

Contents

1	Introduction	2
2	Notation and Optimisation Setup	2
3	Supply and Demand Curves	3
3.1	Supply Curve	3
3.2	Demand Curve	3
4	Problem Formulation	3
4.1	Objective Function	4
4.2	Constraints	4
5	Implementation	5
5.1	Step 1: Reading Data from Excel	5
5.2	Step 2: Preparing Data for Pyomo	5
5.3	Step 3: Creating the Pyomo Model	5
5.3.1	Defining Variables	6
5.4	Step 4: Objective Function and Constraints	6
5.4.1	Objective Function	6
5.4.2	Constraints	6
5.5	Step 5: Solving the Model	7
5.6	Step 6: Displaying Results and Extracting Dual Values	7
6	Steps to Run	8
7	Conclusion	9

1 Introduction

In this document, we present a step-by-step guide to implementing a market-clearing optimization model using Python and Pyomo. The purpose of this model is to determine optimal generation and consumption schedules in an electric power market while maximizing social welfare. The key steps include:

1. Reading input data from an Excel file.
2. Preparing the data for model formulation.
3. Defining the Pyomo model, including sets, parameters, decision variables, the objective function, and constraints.
4. Solving the model using the Gurobi solver.
5. Extracting and displaying the dual values (shadow prices) of key constraints.

This comprehensive guide is intended for researchers and graduate students interested in energy economics and optimization techniques.

2 Notation and Optimisation Setup

In our market model, market participants either supply or consume electric energy. Their offers are characterized by:

- **Quantity (P):** The energy amount in megawatt-hours (MWh).
- **Price (λ):** The price in euros per megawatt-hour (€/MWh).

The market is divided into two sides: **Demand Side:**

Supply Side: Offers by generators, characterized by a maximum capacity $\bar{P}_g^G$ and a cost C_g^G .

Demand Side: Offers by consumers, characterized by a maximum consumption $\bar{P}_d^D$ and a utility U_d .

The decision variables are:

- **Generation Schedule (P^G):** The amount of power generated by each supply-side agent.
- **Consumption Schedule (P^D):** The amount of power consumed by each demand-side agent.

This section sets the stage for the mathematical formulation of the market-clearing problem.

3 Supply and Demand Curves

This section describes the empirical data for the market model. We represent the supply and demand curves using tables that list the offers made by various market participants.

3.1 Supply Curve

The supply side consists of several companies that may submit multiple supply blocks. Each block is characterized by its offered energy (in MWh) and price (in €/MWh). These offers are sorted by the merit order; that is, lower-cost offers are given priority in the scheduling process. Table 1 below provides an example of the supply curve.

Company	Type	id	P_g^G (MW)	C_g^G (€/MWh)
C1	Supply	G1	120	0
C2	Supply	G2	50	0
C3	Supply	G3	200	15
C1	Supply	G4	400	30
C4	Supply	G5	60	32.5
C4	Supply	G6	50	34
C4	Supply	G7	60	36
C5	Supply	G8	100	37.5
C5	Supply	G9	70	39
C5	Supply	G10	50	40
C1	Supply	G11	70	60
C1	Supply	G12	45	70
C6	Supply	G13	50	100
C6	Supply	G14	60	150
C6	Supply	G15	50	200

Table 1: Supply Curve for the Auction (Total of 1435 MWh)

3.2 Demand Curve

Similarly, the demand side is characterized by consumption offers from various companies. Each demand offer specifies the maximum quantity that can be consumed and the corresponding utility value. Higher-paying demand offers (i.e., those with higher utility values) are prioritized. Table 2 illustrates an example of the demand curve.

4 Problem Formulation

In this section, we describe the mathematical formulation of the market-clearing problem.

Company	Type	id	P_d^D (MW)	U_d (€/MWh)
D1	Demand	D1	250	200
D2	Demand	D2	300	180
D3	Demand	D3	120	160
D4	Demand	D4	80	140
D5	Demand	D5	40	120
D6	Demand	D6	70	100
D7	Demand	D7	60	80
D8	Demand	D8	45	60
D9	Demand	D9	30	40
D10	Demand	D10	35	20

Table 2: Demand Curve for the Auction

4.1 Objective Function

The objective is to maximize social welfare, which is defined as the difference between the total utility obtained from electricity consumption and the total generation cost. Mathematically, the objective function is given by:

$$\text{Maximize } SW = \sum_{d=1}^{N_d} U_d p_d^D - \sum_{g=1}^{N_g} C_g p_g^G \quad (1a)$$

This formulation seeks the optimal trade-off between consumers' willingness to pay and the generators' cost of production.

4.2 Constraints

The model is subject to the following constraints:

- **Power Balance:**

The total demand must equal the total generation:

$$\sum_{d=1}^{N_d} p_d^D - \sum_{g=1}^{N_g} p_g^G = 0 \quad (1b)$$

- **Generation and Consumption Limits:**

Each generator and consumer is subject to their respective capacity limits:

$$0 \leq p_g^G \leq \bar{P}_g^G, \quad 0 \leq p_d^D \leq \bar{P}_d^D \quad (1c, 1d)$$

For a minimisation formulation, one can simply take the negative of the objective function without altering the constraints.

5 Implementation

This section provides a detailed explanation of the Python code used to implement the market-clearing model with Pyomo.

5.1 Step 1: Reading Data from Excel

Data is stored in an Excel file (`market_clearing_data.xlsx`) with two sheets: one for generators and one for demands. The code below demonstrates how to read the data using the `pandas` library.

```
1 import pandas as pd
2 excel_file = "market_clearing_data.xlsx"
3 generators_df = pd.read_excel(excel_file, sheet_name="Generators")
4 demands_df = pd.read_excel(excel_file, sheet_name="Demands")
```

Listing 1: Reading Data from Excel

Explanation: This block reads the Excel file and stores the data in two Pandas DataFrames. The data will be later converted into Python lists and dictionaries for use in Pyomo.

5.2 Step 2: Preparing Data for Pyomo

The data is then converted from DataFrames to Python objects:

```
1 generators = generators_df["Generator"].tolist()
2 PGmax = dict(zip(generators_df["Generator"], generators_df["PGmax"]))
3 C = dict(zip(generators_df["Generator"], generators_df["Cost"]))
4
5 demands = demands_df["Demand"].tolist()
6 L = dict(zip(demands_df["Demand"], demands_df["L"]))
7 U = dict(zip(demands_df["Demand"], demands_df["U"]))
```

Listing 2: Preparing Data for Pyomo

Explanation: The generators and demands are converted into lists while their respective parameters (maximum generation, cost, limits, and utility) are stored in dictionaries. This structure simplifies the subsequent model definition in Pyomo.

5.3 Step 3: Creating the Pyomo Model

We then define the Pyomo model by setting up the sets, parameters, and decision variables.

```
1 from pyomo.environ import *
2
3 model = ConcreteModel()
4
5 # Define sets for generators and demands
6 model.g = Set(initialize=generators)
7 model.d = Set(initialize=demands)
8
9 # Define parameters using the dictionaries
```

```

10 model.PGmax = Param(model.g, initialize=PGmax)
11 model.C = Param(model.g, initialize=C)
12 model.L = Param(model.d, initialize=L)
13 model.U = Param(model.d, initialize=U)

```

Listing 3: Creating the Pyomo Model

Explanation: A concrete Pyomo model is created and the sets (g for generators and d for demands) are initialized. The parameters are then declared to link each set with its corresponding numerical data.

5.3.1 Defining Variables

The decision variables for generation and demand are defined as follows:

```

1 model.p_G = Var(model.g, within=NonNegativeReals) # Generation per
    generator
2 model.p_D = Var(model.d, within=NonNegativeReals) # Demand per demand
    point

```

Listing 4: Defining Variables

Explanation: The variables represent the power output of generators and the power consumption of demand points. Both are restricted to non-negative real numbers.

5.4 Step 4: Objective Function and Constraints

5.4.1 Objective Function

We maximize social welfare using the following objective function:

```

1 def objective_rule(model):
2     return sum(model.U[d] * model.p_D[d] for d in model.d) - \
3         sum(model.C[g] * model.p_G[g] for g in model.g)
4 model.objective = Objective(rule=objective_rule, sense=maximize)

```

Listing 5: Defining the Objective Function

Explanation: This function calculates the difference between the total utility (from consumption) and the total cost (from generation), thereby representing the social welfare.

5.4.2 Constraints

We add constraints to the model to enforce generator capacity, demand limits, and power balance.

```

1 # Generator capacity constraint
2 def generator_capacity_rule(model, g):
3     return model.p_G[g] <= model.PGmax[g]
4 model.generator_capacity = Constraint(model.g, rule=
    generator_capacity_rule)
5
6 # Demand capacity constraint
7 def demand_capacity_rule(model, d):

```

```

8     return model.p_D[d] <= model.L[d]
9 model.demand_capacity = Constraint(model.d, rule=demand_capacity_rule)
10
11 # Power balance constraint
12 def balance_rule(model):
13     return sum(model.p_D[d] for d in model.d) == sum(model.p_G[g] for g in
14               model.g)
15 model.balance = Constraint(rule=balance_rule)

```

Listing 6: Defining Constraints

Explanation: Each constraint is defined by a Python function:

- The generator capacity constraint ensures that no generator produces more than its maximum capacity.
- The demand capacity constraint ensures that consumption does not exceed the offered limit.
- The power balance constraint guarantees that total generation equals total consumption.

5.5 Step 5: Solving the Model

We now solve the model using the Gurobi solver. Additionally, we declare a suffix to capture dual values (shadow prices).

```

1 solver = SolverFactory("gurobi") # Ensure Gurobi is installed and
   licensed
2 model.dual = Suffix(direction=Suffix.IMPORT) # For dual variables
3 results = solver.solve(model, tee=True)

```

Listing 7: Solving the Model

Explanation: The solver is invoked to solve the optimization problem. The `Suffix` object is used to import dual information from the solver, which will be used later to interpret the shadow prices associated with the constraints.

5.6 Step 6: Displaying Results and Extracting Dual Values

Finally, we print the optimal solution, including generation, demand, and market-clearing price. We also extract and display the dual values for the generator capacity and demand capacity constraints.

```

1 print(f"{'='*10} Optimal Solution {'='*10}")
2 print("Social Welfare (SW):", model.objective())
3 for g in model.g:
4     print(f"Generation by {g}: {model.p_G[g].value} MW")
5 for d in model.d:
6     print(f"Demand by {d}: {model.p_D[d].value} MW")
7
8 # Market clearing price from the power balance constraint

```



```

9  if model.balance in model.dual:
10     print("Market Clearing Price of Electricity:", model.dual[model.
        balance])
11 else:
12     print("No dual value found for the balance constraint.")
13
14 # Extract and display dual values for generator capacity constraints
15 print("Dual values for generator capacity constraints:")
16 for g in model.g:
17     dual_value = model.dual.get(model.generator_capacity[g])
18     if dual_value is not None:
19         print(f"Generator {g}: {dual_value}")
20     else:
21         print(f"Generator {g}: No dual value found.")
22
23 # Extract and display dual values for demand capacity constraints
24 print("Dual values for demand capacity constraints:")
25 for d in model.d:
26     dual_value = model.dual.get(model.demand_capacity[d])
27     if dual_value is not None:
28         print(f"Demand {d}: {dual_value}")
29     else:
30         print(f"Demand {d}: No dual value found.")

```

Listing 8: Displaying Results and Dual Values

Explanation: This block prints the optimal social welfare, the generation and demand levels, and extracts the dual values (shadow prices) for each constraint:

- The dual of the power balance constraint represents the market-clearing price.
- The dual values of the generator and demand capacity constraints provide marginal values, indicating how the objective would change if the respective limits were relaxed.

6 Steps to Run

To run the model, follow these steps:

1. Save the Python script (e.g., `market_clearing_model.py`) and the Excel data file (`market_clearing_data.xlsx`) in the same directory.
2. Run the script from the command line:

```
1 python market_clearing_model.py
```

3. Examine the output in the console for the optimal social welfare, generation and demand levels, and the dual values (shadow prices) of the constraints.

7 Conclusion

This document presented a detailed guide for implementing a market-clearing optimization model using Pyomo. In addition to determining optimal generation and consumption schedules, the model extracts dual values that offer insight into the marginal worth of the constraints. This approach can be extended with additional constraints and data to model more complex market scenarios. Researchers and students are encouraged to build upon this model to explore various aspects of energy market economics.